

Desafío Técnico Backend

¡Felicidades por haber llegado a esta etapa!

Queremos agradecerte por el tiempo que vienes dedicándole a nuestro proceso de selección. Sabemos que cada una de las etapas demanda esfuerzo y valoramos la entrega que le estás poniendo a cada una de ellas. Ahora, llegó el momento de demostrar tus conocimientos y experiencia en la resolución de este caso.

Esta prueba busca evaluar:

- Diseño de API REST y estructura limpia de código
- Uso de herramientas modernas de desarrollo en Python
- Pensamiento técnico y capacidad de justificación
- Buenas prácticas: testing, Docker, linters, validaciones
- Tiempo estimado: 4-6 horas. Si no puedes cubrir todo, prioriza lo principal y documenta lo que dejarías como pendiente.

Sobre el Desafío

Requisitos:

- Lenguaje: Python
- Framework de API: FastAPI
- Base de Datos: Implementar una base de datos real de tu preferencia.
- Pruebas: pytest o unittest
- Linter: flake8 o pylint
- Formateo de Código: black
- Docker: Configuración de un contenedor Docker para ejecutar la aplicación.

Detalles:

1. Casos de uso

a. Casos de uso básicos:

- Crear, obtener, actualizar y eliminar listas de tareas.
- Crear, obtener, actualizar y eliminar tareas dentro de una lista.
- Cambiar el estado de una tarea.
- Listar todas las tareas de una lista con filtros por estado o prioridad y un campo extra donde se indique el porcentaje de completitud.

b. Casos de uso que suman puntos:

- Estos casos de uso no son obligatorios, pero suman puntos si los realizas.
- Login y autenticación (bonus): Implementación opcional de JWT para proteger endpoints.
- Asignación de tareas: Asignar un usuario responsable a cada tarea.
- Notificación ficticia: Simulación de envío de invitación a usuarios por email (no real).

2. Estructura del Proyecto

- Estructura limpia por capas (Domain, Application/UseCases, Infrastructure).
- Tipado fuerte con Pydantic.
- Manejo de errores con excepciones personalizadas.
- Validaciones de negocio.
- Testing unitario y de integración con pytest.

- f. Linters (flake8, ruff) y formateo (black, isort).
- g. Dockerfile (multistage si aplica) y docker-compose.
- h. README completo + archivo DECISION_LOG.md explicando decisiones técnicas.

3. Testing

- a. Implementar pruebas unitarias y de integración utilizando pytest.
- b. Las pruebas deben cubrir un 75% del proyecto
- c. Incluir un archivo pytest.ini para configurar pytest.

4. Linter y Formateo de Código

- a. Configurar flake8 como linter para el proyecto.
- b. Configurar black como formateador de código.
- c. Incluir un archivo .flake8 con configuraciones (por ejemplo, ignorar ciertas reglas).

5. Docker

- a. Proveer un Dockerfile para crear la imagen de Docker que contenga la aplicación. Incluir un archivo docker-compose.yml opcional para facilitar el levantamiento de la aplicación.

6. README

- a. El archivo README.md debe incluir:
- b. Descripción del proyecto.
- c. Instrucciones para configurar el entorno local.
- d. Instrucciones para ejecutar la aplicación en Docker.
- e. Instrucciones para correr las pruebas.

Al finalizar, recuerda enviar tu repositorio en respuesta al correo electrónico.

¡Muchos éxitos!